

App Store wrapper readiness

Budger | Read-only technical audit

CURRENT STATUS: NOT APP STORE-READY YET

The repository has a useful compliance scaffold, but it is not currently a buildable iOS application.

What is already in good shape

- Production frontend is reachable at <https://budger.app>
- Production backend health endpoint responds successfully
- Production frontend build passes
- Privacy policy includes crash-report consent, data export, account deletion, and receipt/photo handling
- In-app data export endpoint exists
- Account deletion flow exists
- Crash reporting is opt-in by default
- Camera and photo-library browser functionality exists
- Backend has APNs-related code
- An initial PrivacyInfo.xcprivacy file exists
- An initial Expo-style iOS configuration exists
- The web app has notification permission and push subscription logic

What is missing before a native build can be produced

The export/ios directory currently contains only:

- app.config.ts
- PrivacyInfo.xcprivacy

It does not contain:

- An Xcode project
- An Expo project with package.json
- Native iOS source files
- CocoaPods configuration
- EAS configuration
- App icon assets
- Splash-screen assets
- A valid EAS project ID
- Signing or provisioning configuration
- A buildable .ipa

The configuration references these missing files:

```
export/ios/assets/icon.png
export/ios/assets/splash.png
export/ios/assets/adaptive-icon.png
```

It also contains the placeholder:

REPLACE_WITH_EAS_PROJECT_ID

The current iOS folder is therefore a preparation template, not something that can be opened in Xcode or uploaded to App Store Connect.

Important technical concern: this is currently a web app, not a native app

The application uses:

- React/Vite
- Browser localStorage
- Browser service workers
- Browser Notification
- Browser PushManager
- VAPID web push
- Browser camera/file APIs

A simple WKWebView wrapper would not automatically provide:

- Native APNs push notifications
- Native background tasks
- Native camera/photo integrations
- Native biometric authentication
- Reliable iOS push-token registration
- Native offline behavior
- Native Live Activities

The existing backend APNs code is not enough by itself. A native iOS client would need to register an APNs token and send that token to the backend through a native bridge or native app code.

Apple could also reject a basic website wrapper under the minimum-functionality guideline if it does not provide meaningful native-app value.

Configuration items that need correction or verification

The current scaffold includes declarations that are not yet backed by native implementation:

- aps-environment
- Background fetch
- Remote notifications
- Face ID usage description
- Native camera/photo permission declarations
- Live Activity comments

These should only remain if the final native app actually implements those capabilities. In particular:

- Face ID is described as a future feature but is not implemented.
- Background fetch is declared, but the current application uses browser/service-worker behavior rather than a native background task.
- APNs is declared, but native token registration is not present.
- The notification system currently uses web push rather than native APNs.

Leaving unsupported declarations in the final binary could create App Review questions or privacy inconsistencies.

Privacy and App Store compliance status

The privacy foundation is promising but still needs final verification.

Already present:

- Privacy policy
- Data export
- Account deletion request flow
- Crash-report consent
- Privacy manifest draft
- Camera/photo explanations

Still required:

- App Store Connect privacy questionnaire
- Accurate declaration of email address, name, financial information, photos, device identifiers, and crash/performance data
- Confirmation of whether Sentry data is linked to identity
- Confirmation of whether receipt images are retained and for how long
- Confirmation of all third-party processors
- Final privacy manifest generated from the actual native dependencies
- App Review test account and review notes
- Verification that account deletion works from the submitted iOS build

The privacy manifest should also be reviewed against the actual native dependency tree. It is currently written for an anticipated Expo/native app, not an existing compiled app.

Typecheck status

The production frontend build passes.

The full frontend typecheck currently reports unrelated existing errors in:

- HouseholdDonutChart.tsx
- Household.tsx
- usePullToRefresh.ts
- Duplicate translation keys in i18n.ts

Those should be fixed before treating the repository as release-clean, although they do not prevent the current Vite production build.

Overall assessment

I would classify the project as:

- Web/PWA production readiness: Good
- Privacy-feature readiness: Mostly good
- Native iOS wrapper readiness: Early scaffold only
- App Store upload readiness: Not ready
- Estimated remaining work: A proper native-wrapper implementation, not merely packaging

Once the native approach is selected - Expo/React Native or Capacitor - the wrapper can be prepared with native push/camera/permissions, an Xcode project, assets, signing configuration, and an App Store submission workflow.